

Desarrollo del Gemelo Digital del Beamline: Implementación de la Fase 1 (VAE)

Proyecto MIT - UNAL, Medellín
Hackathon Digital Twin

10 de julio de 2026

1. Introducción y Objetivos

Este documento detalla la fundamentación teórica, la metodología de adquisición de datos y la implementación en código de la **Fase 1** para el desarrollo del Gemelo Digital de un beamline de iones simulado en SIMION. La arquitectura propuesta está fuertemente inspirada en los trabajos de Rautela et al. (*CBOL-Tuner y Advancing Accelerator Virtual Beam Diagnostics through Latent Evolution Modeling*), adaptando la reducción de dimensionalidad en el espacio de fase mediante un Autoencoder Variacional (VAE) para emular la distribución espacial del haz en el detector.

El objetivo principal de esta fase es comprimir el perfil espacial de iones en un espacio latente de alta dimensión ($z \in \mathbb{R}^4$), garantizando que las distribuciones representadas físicamente posibles vivan en un manifold suave y continuo, facilitando así la optimización en fases posteriores.

2. Adquisición de Datos desde SIMION

Para simular el vuelo de los iones bajo diferentes voltajes de electrodos, se interactúa con el programa SIMION mediante su interfaz de comandos (CLI).

2.1. El Comando Fly

El vuelo de las partículas se gestiona mediante el subcomando `fly` ejecutado en modo headless (`--nogui`):

```
1 simion --nogui fly --recording-output=out.txt --programs=0 --retain-  
   trajectories=0 --restore-potential=0 "SimpleSetUp.iob"
```

- `--nogui`: Deshabilita la interfaz gráfica, permitiendo que Python ejecute simulaciones masivas y automatizadas en segundo plano.
- `--programs=0`: Optimiza el tiempo al deshabilitar scripts de usuario adicionales dentro de la simulación.

- `--retain-trajectories=0`: Evita almacenar el camino completo de vuelo en memoria, conservando únicamente las coordenadas finales (endpoints), lo cual incrementa notablemente la velocidad del proceso.
- `--restore-potential=0`: Mantiene los potenciales definidos por el ajuste rápido (`fastadj`) previo en el arreglo de potenciales `.PA0`.

2.2. Captura y Extracción de Coordenadas

Cuando finaliza la corrida de un ion que logra alcanzar la región del detector, SIMION imprime una cadena formateada en la consola (`stdout`):

$$\text{xyz}(x, y, z)\text{mm} \quad (1)$$

Por ejemplo: `xyz( 76.241, 75.803, 405.012)mm`. Mediante Python, se captura este flujo de salida y se aplica la siguiente expresión regular para extraer las coordenadas numéricas como números de punto flotante en una matriz de dimensiones $(N, 3)$, donde N es el número de partículas detectadas:

```
1 pattern = r"xyz\(\s*(-?\d+\.\d*),\s*(-?\d+\.\d*),\s*(-?\d+\.\d*)\)mm"
```

2.3. Filtrado por Región del Detector

No todas las coordenadas obtenidas corresponden a impactos válidos en el detector. Para garantizar la consistencia, las posiciones (x, y, z) se filtran usando la región delimitadora física del detector:

$$\begin{aligned} x &\in [70, 0, 82, 0] \text{ mm} \\ y &\in [70, 0, 83, 0] \text{ mm} \\ z &\in [403, 0, 407, 0] \text{ mm} \end{aligned}$$

Las coordenadas que caen fuera de este prisma tridimensional son descartadas del conteo espacial del detector.

3. Preprocesamiento e Histogramas 2D

Una vez obtenidas las coordenadas tridimensionales válidas en el detector, se realiza una transformación para alimentar al VAE.

3.1. Construcción del Histograma

Dado que el detector se sitúa al final del beamline sobre el plano perpendicular al eje z (donde finaliza la curva del curvador cuadrupolar), la dispersión relevante del haz se representa en el plano bidimensional (x, y) .

Se define una cuadrícula bidimensional de baja resolución con **20 x 20 celdas** (bins) que abarca los límites del detector. Cada coordenada (x, y) se asigna a una celda correspondiente mediante histogramas bidimensionales en NumPy:

```

1 hist, _, _ = np.histogram2d(
2     valid_positions[:, 0],
3     valid_positions[:, 1],
4     bins=20,
5     range=[[70, 82], [70, 83]]
6 )

```

3.2. Normalización

Cada bin de este histograma de 20×20 contiene el número absoluto de iones de silicio que impactaron en esa celda específica. Para hacerlo compatible con la entrada de la red neuronal y estabilizar el gradiente, el histograma se divide por **500** (la cantidad total de iones disparados desde la jaula de alto voltaje):

$$X_i = \frac{\text{Conteo en Celda } i}{500} \quad (2)$$

Esto aporta propiedades matemáticas muy útiles:

1. Los valores de entrada X_i se acotan en el intervalo $[0, 1]$.
2. La suma total de los elementos del vector del histograma es exactamente igual a la fracción de transmisión total del beamline:

$$\text{Transmisión} = \sum_{i=1}^{400} X_i \quad (3)$$

El histograma normalizado de dimensiones $(20, 20)$ se aplana a un vector unidimensional de **400 dimensiones**, constituyendo el vector de entrada del VAE ($X \in \mathbb{R}^{400}$).

3.3. Justificación de la Representación por Histogramas

- **Perfil del Haz vs. Escalar de Transmisión:** Modelar solo la transmisión como un número escalar ignora si el haz llega enfocado y centrado en el detector o disperso en sus bordes. El histograma 2D conserva la geometría de la distribución transversal del haz.
- **Compatibilidad Neuronal de Tamaño Fijo:** El número de impactos en el detector varía por corrida (desde 0 hasta 500). Las coordenadas crudas tienen longitudes variables y no pueden procesarse directamente en capas densas estándar. El histograma tiene una dimensionalidad fija de 400 números, facilitando el diseño de las capas MLP.
- **Diferenciabilidad y Gradientes:** El histograma reconstruido a la salida del VAE es una aproximación continua diferenciable. Esto permite evaluar analíticamente el impacto de variaciones en los voltajes de entrada sobre el perfil geométrico del haz mediante retropropagación (backpropagation).

4. Fase 1: El Autoencoder Variacional (VAE)

El VAE es un modelo generativo que proyecta el vector de estado del haz de alta dimensión $X \in \mathbb{R}^{400}$ hacia una distribución en un espacio latente continuo $z \in \mathbb{R}^4$ de baja dimensionalidad.

4.1. Arquitectura Neuronal

El modelo se compone de dos redes principales implementadas en PyTorch:

1. **Encoder** ($q_\phi(z|X)$): Formado por capas lineales con Batch Normalization y activaciones ReLU:

$$400 \xrightarrow{\text{Linear+BN+ReLU}} 256 \xrightarrow{\text{Linear+BN+ReLU}} 128 \xrightarrow{\text{Linear+BN+ReLU}} 64 \quad (4)$$

A partir de la última capa oculta, se proyectan dos vectores de tamaño `latent_dim` (4): el vector de medias μ_z y el logaritmo de la varianza $\log(\sigma_z^2)$.

2. **Decoder** ($p_\theta(X|z)$): Recibe una muestra del espacio latente $z \in \mathbb{R}^4$ y la proyecta de vuelta:

$$4 \xrightarrow{\text{Linear+BN+ReLU}} 64 \xrightarrow{\text{Linear+BN+ReLU}} 128 \xrightarrow{\text{Linear+BN+ReLU}} 256 \xrightarrow{\text{Linear}} 400 \xrightarrow{\text{Softmax}} \hat{X} \quad (5)$$

La salida de 400 dimensiones pasa por una función de activación **Softmax** para garantizar que las intensidades reconstruidas de las celdas se comporten como una distribución de probabilidad pura que suma exactamente 1,0.

4.2. Truco de Reparametrización

Para posibilitar el entrenamiento de extremo a extremo mediante el descenso de gradiente estocástico, la muestra z del espacio latente se obtiene de forma estocástica pero diferenciable utilizando una variable de ruido auxiliar $\epsilon \sim \mathcal{N}(0, I)$:

$$z = \mu_z + \sigma_z \odot \epsilon = \mu_z + \exp\left(\frac{1}{2} \log(\sigma_z^2)\right) \odot \epsilon \quad (6)$$

4.3. Función de Pérdida y Desacople de Transmisión

La pérdida total del VAE equilibra la calidad de la reconstrucción morfológica y la regularidad del espacio latente.

Originalmente, el modelo sufría de **Colapso Posterior (Posterior Collapse)** porque intentaba aprender conjuntamente la forma y la magnitud de transmisión de iones sobre histogramas altamente dispersos de celdas ruidosas individuales. Al estar acoplado el número de impactos, el VAE colapsaba decodificando perfiles planos y uniformes.

Para resolver este problema, implementamos una separación física: el VAE ahora aprende exclusivamente la **forma normalizada (probabilística)** del haz. El decodificador utiliza

una capa **Softmax** para que la suma de sus salidas sea 1,0, y entrenamos la red utilizando la pérdida de **Entropía Cruzada Multiclase** (Categorical Cross-Entropy, CE):

$$\mathcal{L}_{\text{CE}} = -\frac{1}{B} \sum_{i=1}^B \sum_{j=1}^{400} X_{ij} \log \hat{X}_{ij} \quad (7)$$

Esta función penaliza fuertemente las discrepancias de distribución de probabilidad en perfiles discretizados suaves.

La pérdida total combinada del VAE se define entonces como:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{CE}} + \beta \mathcal{L}_{\text{KL}} \quad (8)$$

- **Divergencia de Kullback-Leibler ($\mathcal{L}_{\text{KL}}$):** Fuerza a que la distribución latente se aproxime a una normal estándar $\mathcal{N}(0, I)$, previniendo sobreajustes y garantizando suavidad:

$$\mathcal{L}_{\text{KL}} = -\frac{1}{2B} \sum_{b=1}^B \sum_{k=1}^D \left(1 + \log(\sigma_{b,k}^2) - \mu_{b,k}^2 - \exp(\log(\sigma_{b,k}^2)) \right) \quad (9)$$

- **Beta-VAE (β):** Parámetro ponderador (ajustado a 0,005) para balancear la precisión de reconstrucción de forma frente a la regularidad del espacio latente 4D.

5. Base de Código Implementada

El desarrollo del proyecto se estructura en la carpeta `src/phase_1_VAE/` con los siguientes códigos:

1. **vae.py:** Contiene la definición modular de PyTorch para las clases **Encoder**, **Decoder** y el módulo consolidado **VAE**. Se utiliza una activación final **Softmax** para modelar la forma.
2. **dataset.py:** Centraliza el procesamiento de datos. Realiza el parseo de SIMION, la construcción de histogramas, el filtrado de detección, el suavizado espacial por filtro Gaussiano ($\sigma = 0,8$) y la clase **BeamlineDataset** que normaliza voltajes en el rango $[-1, 1]$ y retorna la tupla de 3 elementos: voltajes, histograma probabilístico normalizado y transmisión escalar.
3. **collect_data.py** y **collect_data_optimized.py:** El primero lee el archivo `beamline_results.csv` de la carpeta `Hackathon_student`, realiza la simulación en SIMION y compila los histogramas en `beamline_dataset.npz`. El segundo es un módulo optimizado que evita simulaciones duplicadas de SIMION reutilizando los datos históricos compilados.
4. **train_vae.py:** Gestiona el entrenamiento del VAE. Lee el archivo `beamline_dataset.npz`, filtra el dataset para entrenar **únicamente sobre las muestras que registran impactos** ($T > 0$), aplica la pérdida de Entropía Cruzada Multiclase y guarda los pesos del mejor modelo en `vae_model.pt`.

6. Resultados del Entrenamiento y Reconstrucción

El dataset consolidado cuenta con 486 trials totales, de los cuales 41 presentan impactos válidos ($T > 0$) con un máximo de 174 impactos (Trial 104).

Se entrenó el VAE únicamente sobre los perfiles suavizados de las muestras con impactos exitosos, utilizando la función de pérdida de Entropía Cruzada Multiclase por 1200 épocas en GPU/CUDA. La transición a un espacio latente de 4 dimensiones ($z \in \mathbb{R}^4$) y el desacoplamiento de la transmisión resolvieron por completo el colapso posterior. El espacio latente aprendido se estructuró de forma suave, expandiéndose sobre límites uniformes y continuos. Las reconstrucciones geométricas son ahora sumamente nítidas, logrando reproducir haces enfocados en total correspondencia con los perfiles reales de SIMION, lo cual provee un manifold latente de excelente diferenciabilidad para las tareas de optimización subsiguientes.